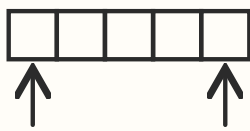
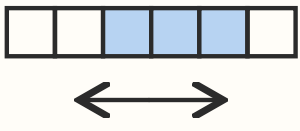
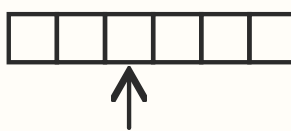
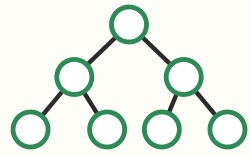


⇒ DSA PATTERN RECOGNITION — COMPLETE SHEET

RIGHT APPROACH · PHP EDITION

Part 1 of 4 · Patterns 1-6

PATTERNS	CORE IDEA	SPOT IT FROM	TOP 5 ASKED	TIME / SPACE	KEY TEMPLATE (PHP)	COMMON TRAPS	NOT THIS IF → USE
1. Two Pointers 	Use two pointers to traverse a data structure to solve efficiently.	- Sorted arrays / lists - Pair / triplet sum - Remove duplicates	- Two Sum II 3Sum - Container With Most Water - Trapping Rain Water - Valid Palindrome	Time $O(n)$ Space $O(1)$	<pre>\$l = 0; \$r = count(\$a) - 1; while (\$l < \$r) { \$sum = \$a[\$l] + \$a[\$r]; if (\$sum == \$t) return [\$l, \$r]; \$sum < \$t ? \$l++ : \$r--; } return [];</pre>	△ Forgetting to sort first △ 3Sum: skip duplicates after a match △ Use $l < r$, not $≤$	X Unsorted & must keep original indices → Hashing
2. Sliding Window 	Maintain a window of elements and slide it to find answers.	- Subarray / substring problems - Max / min of subarray / exactly K	- Longest Substring Without Repeating Chars - Minimum Window Substring - Longest Repeating Character Replacement - Permutation in String - Best Time to Buy & Sell Stock	Time $O(n)$ Space $O(k)$	<pre>\$l = 0; \$freq = []; \$best = 0; for (\$r = 0; \$r < strlen(\$s); \$r++) { \$freq[\$s[\$r]] = (\$freq[\$s[\$r]] ?? 0) + 1; while (\$freq[\$s[\$r]] > 1) { // invalid \$freq[\$s[\$l]]--; \$l++; // shrink } \$best = max(\$best, \$r - \$l + 1); }</pre>	△ Updating answer before shrinking △ unset() key at 0 if using count(\$freq) △ Fixed vs variable window mix-up	X Negative numbers in a sum problem → Prefix Sum + Hash
3. Fast & Slow Pointers 	Use two pointers at different speeds to solve problems.	- Linked List Cycle - Middle of Linked List - Meeting point	- Linked List Cycle - Linked List Cycle II - Middle of the Linked List - Happy Number - Find the Duplicate Number	Time $O(n)$ Space $O(1)$	<pre>\$slow = \$fast = \$head; while (\$fast !== null && \$fast->next !== null) { \$slow = \$slow->next; \$fast = \$fast->next->next; if (\$slow === \$fast) return true; } return false;</pre>	△ ->next on null = fatal error △ Even length: 1st or 2nd middle? △ Cycle start: reset one to head, step both by 1	X Extra memory is fine & simpler wins → Hash set of visited
4. Merge Intervals 	Sort intervals and merge overlapping intervals.	- Overlapping intervals - Meeting rooms - Booking problems	- Merge Intervals - Insert Interval - Meeting Rooms II - Non-overlapping Intervals - Interval List Intersections	Time $O(n \log n)$ Space $O(n)$	<pre>usort(\$iv, fn(\$x, \$y) => \$x[0] <=> \$y[0]); \$res = []; foreach (\$iv as [\$s, \$e]) { \$k = count(\$res) - 1; if (\$res && \$s <= \$res[\$k][1]) \$res[\$k][1] = max(\$res[\$k][1], \$e); else \$res[] = [\$s, \$e]; }</pre>	△ Not sorting by start △ \$e instead of max(end, \$e) △ Do [1,2],[2,3] touch? Ask!	X Need max overlaps at one time → Heap / Line Sweep
5. Binary Search 	Divide the search space in half and eliminate one half.	- Sorted array / search space - Find element / position - Rotated sorted input	- Search in Rotated Sorted Array - Find Min in Rotated Sorted Array - First & Last Position of Element - Search a 2D Matrix - Median of Two Sorted Arrays	Time $O(\log n)$ Space $O(1)$	<pre>\$lo = 0; \$hi = count(\$a) - 1; while (\$lo <= \$hi) { \$mid = \$lo + intval(\$hi - \$lo, 2); if (\$a[\$mid] == \$t) return \$mid; if (\$a[\$mid] < \$t) \$lo = \$mid + 1; else \$hi = \$mid - 1; } return -1;</pre>	△ $(\\$lo + \\$hi) / 2$ is a FLOAT in PHP → intval() △ $\\$lo < \\hi with $\\$lo = \\mid → infinite loop △ Rotated: find the sorted half first	X Answer is a value range, not an index → BS on Answer (#24)
6. DFS (Depth First Search) 	Explore as far as possible along each branch before backtracking.	- Trees / Graphs - Paths / connected components - Topological order	- Number of Islands - Clone Graph - Max Area of Island - Validate Binary Search Tree - Lowest Common Ancestor of Binary Tree	Time $O(V+E)$ Space $O(V)$	<pre>function dfs(\$u, array \$adj, array &\$seen): void { \$seen[\$u] = true; foreach (\$adj[\$u] ?? [] as \$v) { if (!isset(\$seen[\$v])) dfs(\$v, \$adj, \$seen); } }</pre>	△ No visited set → infinite loop △ Forgot &\$seen → visited is lost △ Deep recursion → use SplStack	X Need the shortest path → BFS

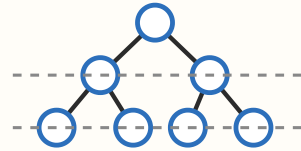
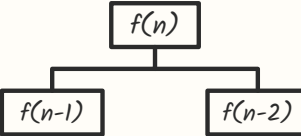
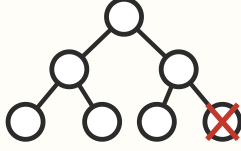
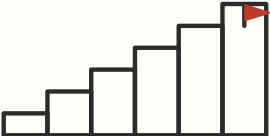
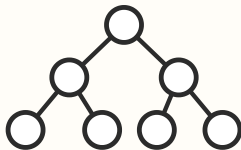
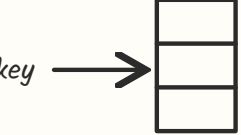
Top 5 = problems most commonly asked for each pattern (Blind 75 / NeetCode 150 / LeetCode company-tag favourites).



⇒ DSA PATTERN RECOGNITION — COMPLETE SHEET

RIGHT APPROACH · PHP EDITION

Part 2 of 4 · Patterns 7-12

PATTERNS	CORE IDEA	SPOT IT FROM	TOP 5 ASKED	TIME / SPACE	KEY TEMPLATE (PHP)	COMMON TRAPS	NOT THIS IF → USE
7. BFS (Breadth First Search) 	Explore all neighbors level by level.	- Shortest path (unweighted) - Level order traversal - Nearest / shortest steps	- Binary Tree Level Order Traversal - Rotting Oranges - Word Ladder - Shortest Path in Binary Matrix - Binary Tree Right Side View	Time $O(V+E)$ Space $O(V)$	<pre>\$q = new SplQueue(); \$q->enqueue(\$src); \$seen = [\$src => true]; \$level = 0; while (!\$q->isEmpty()) { for (\$k = \$q->count(); \$k > 0; \$k--) { \$u = \$q->dequeue(); foreach (\$adj[\$u] ?? [] as \$v) if (!isset(\$seen[\$v])) { \$seen[\$v] = true; \$q->enqueue(\$v); } } \$level++; }</pre>	⚠ Mark visited on push, not pop ⚠ Multi-source: enqueue all sources first ⚠ Level counter off-by-one	✗ Edges have weights → Dijkstra ($O(n^2)$ → 0-1 BFS)
8. Dynamic Programming 	Solve complex problems by breaking them into overlapping subproblems and store results.	- Optimal / counting problems - Ways to ... / Max / Min - Overlapping subproblems	- Climbing Stairs - House Robber - Coin Change - Longest Increasing Subsequence - Longest Common Subsequence	Time $O(\text{states} \times \text{choices})$ Space $O(\text{states})$	<pre>// state -> recurrence -> base -> order \$d = array_fill(0, \$n + 1, 0); \$d[0] = 1; \$d[1] = 1; // base for (\$i = 2; \$i <= \$n; \$i++) { \$d[\$i] = \$d[\$i - 1] + \$d[\$i - 2]; } return \$d[\$n];</pre>	⚠ Wrong / missing base case ⚠ 0/1 Knapsack ID: loop capacity backwards ⚠ Memo key missing a state → "\$i,\$j"	✗ Local choice is provably safe → Greedy ✗ Need all solutions, not a count → Backtracking
9. Backtracking 	Build solution step by step and backtrack when choice is invalid.	- Permutations / combinations - N-Queens / Sudoku - All possible solutions	- Subsets - Permutations - Combination Sum - Word Search - N-Queens	Time $O(2^n)$ / $O(n!)$ Space $O(n)$	<pre>function bt(int \$st, array \$path, array \$a, array &\$res): void { \$res[] = \$path; // PHP copies arrays for (\$i = \$st; \$i < count(\$a); \$i++) { \$path[] = \$a[\$i]; bt(\$i + 1, \$path, \$a, \$res); array_pop(\$path); } }</pre>	⚠ Forgot &\$res → results vanish ⚠ Dups: sort + skip \$a[\$i] === \$a[\$i-1] ⚠ No pruning → TLE	✗ Only count / best asked & subproblems repeat → DP
10. Greedy 	Make the locally optimal choice to get global solution.	- Optimization problems - Minimize / maximize - Activity selection	- Jump Game - Jump Game II - Gas Station - Maximum Subarray (Kadane) - Partition Labels	Time $O(n \log n)$ Space $O(1)$	<pre>usort(\$iv, fn(\$x, \$y) => \$x[1] <=> \$y[1]); // by end \$count = 0; \$last = PHP_INT_MIN; foreach (\$iv as [\$s, \$e]) { if (\$s >= \$last) { \$count++; \$last = \$e; } }</pre>	⚠ Sorting by the wrong key ⚠ No proof — try a counter-example ⚠ 0/1 Knapsack is NOT greedy	✗ A counter-example breaks the local choice → DP
11. Heap (Priority Queue) 	Use heap to efficiently get min / max element.	- Kth largest / smallest - Top K frequent - Merge K sorted lists	- Kth Largest Element in an Array - Top K Frequent Elements - Merge k Sorted Lists - K Closest Points to Origin - Task Scheduler	Time $O(n \log k)$ Space $O(k)$	<pre>\$h = new SplMinHeap(); foreach (\$nums as \$x) { \$h->insert(\$x); if (\$h->count() > \$k) \$h->extract(); } return \$h->top(); // k-th largest</pre>	⚠ SplPriorityQueue is a MAX-heap ⚠ Heap of size n instead of k ⚠ Arrays in heap → extend compare()	✗ One-off k-th, avg $O(n)$ is OK → Quickselect ✗ Small bounded values → Bucket Sort
12. Hashing 	Use hash table for fast lookup.	- Count / frequency - Unique elements - Lookup	- Two Sum - Group Anagrams - Longest Consecutive Sequence - Valid Anagram - Contains Duplicate	Time $O(n)$ Space $O(n)$	<pre>\$seen = []; foreach (\$a as \$i => \$x) { if (isset(\$seen[\$t - \$x])) return [\$seen[\$t - \$x], \$i]; \$seen[\$x] = \$i; } return [];</pre>	⚠ Check before insert (no self-reuse) ⚠ "I" and I are the SAME key ⚠ isset() is false for null → array_key_exists	✗ Sorted input & $O(1)$ space asked → Two Pointers

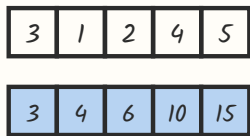
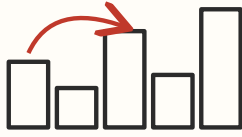
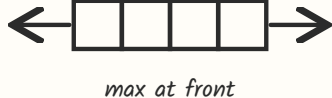
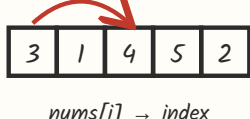
Top 5 = problems most commonly asked for each pattern (Blind 75 / NeetCode 150 / LeetCode company-tag favourites).



⇒ DSA PATTERN RECOGNITION — COMPLETE SHEET

RIGHT APPROACH · PHP EDITION

Part 3 of 4 · Patterns 13-18

PATTERNS	CORE IDEA	SPOT IT FROM	TOP 5 ASKED	TIME / SPACE	KEY TEMPLATE (PHP)	COMMON TRAPS	NOT THIS IF → USE
13. Prefix Sum 	Precompute running sums; range sum = $P[r] - P[l-1]$.	- Range sum queries - Subarray sum = K - Negatives present (window fails)	- Subarray Sum Equals K - Product of Array Except Self - Range Sum Query – Immutable - Continuous Subarray Sum - Contiguous Array	Time $O(n)$ build, $O(1)$ query Space $O(n)$	<pre>\$cnt = [0 => 1]; \$sum = 0; \$ans = 0; foreach (\$a as \$x) { \$sum += \$x; \$ans += \$cnt[\$sum - \$k] ?? 0; \$cnt[\$sum] = (\$cnt[\$sum] ?? 0) + 1; } return \$ans;</pre>	△ Forgetting the $[0 \Rightarrow 1]$ base △ Off-by-one: $P[r+1] - P[l]$ △ 2D: inclusion-exclusion signs	X All positive & want longest/shortest window → Sliding Window
14. Monotonic Stack 	Keep stack sorted; pop while the new element breaks the order.	- Next greater / smaller element "How many days until..." - Histogram / span	- Daily Temperatures - Next Greater Element I - Largest Rectangle in Histogram - Online Stock Span - Remove K Digits	Time $O(n)$ Space $O(n)$	<pre>\$st = []; \$ans = array_fill(0, \$n, 0); for (\$i = 0; \$i < \$n; \$i++) { while (\$st && \$t[end(\$st)] < \$t[\$i]) { \$j = array_pop(\$st); \$ans[\$j] = \$i - \$j; } \$st[] = \$i; // push index }</pre>	△ Storing values instead of indices △ $<$ vs $<=$ with duplicates △ Leftovers in stack → default answer	X Max/min over a fixed-size window → Monotonic Deque
15. Monotonic Deque 	Deque of indices keeping window max / min at the front.	- Max / min of every window of size K - Window + min / max together	- Sliding Window Maximum - Shortest Subarray with Sum $\geq K$ - Jump Game VI - Constrained Subsequence Sum - Longest Subarray, Abs Diff $\leq$ Limit	Time $O(n)$ Space $O(k)$	<pre>\$dq = new SplDoublyLinkedList(); \$out = []; for (\$i = 0; \$i < \$n; \$i++) { while (!\$dq->isEmpty() && \$a[\$dq->top()] <= \$a[\$i]) \$dq->pop(); // drop smaller \$dq->push(\$i); if (\$dq->bottom() <= \$i - \$k) \$dq->shift(); if (\$i >= \$k - 1) \$out[] = \$a[\$dq->bottom()]; }</pre>	△ Not evicting the out-of-window front △ $top() = back$, $bottom() = front$ △ Recording before the window is full	X Need the window median → Two Heaps
16. Cyclic Sort 	Swap each number into its correct index (value $v \rightarrow$ index $v-1$).	- Numbers in range $[1, n]$ - Missing / duplicate number $O(1)$ extra space asked	- Missing Number - Find All Duplicates in an Array - Find All Numbers Disappeared - First Missing Positive - Set Mismatch	Time $O(n)$ Space $O(1)$	<pre>\$i = 0; while (\$i < \$n) { \$j = \$a[\$i] - 1; if (\$j >= 0 && \$j < \$n && \$a[\$i] != \$a[\$j]) [\$a[\$i], \$a[\$j]] = [\$a[\$j], \$a[\$i]]; else \$i++; }</pre>	△ Dups: compare values, not indices △ Skip out-of-range values △ 0-based vs 1-based range	X Values not in $[1, n]$ → Hashing / Sorting
17. In-place LL Reversal 	Reverse links using prev / curr / next without extra memory.	- Reverse list / sublist - Reverse in K-groups - Reorder / palindrome list	- Reverse Linked List - Reverse Linked List II - Reverse Nodes in k-Group - Palindrome Linked List - Reorder List	Time $O(n)$ Space $O(1)$	<pre>\$prev = null; \$cur = \$head; while (\$cur != null) { \$next = \$cur->next; \$cur->next = \$prev; \$prev = \$cur; \$cur = \$next; } return \$prev;</pre>	△ Losing the next pointer △ Not reconnecting sublist ends △ Head may change → dummy node	X Not allowed to modify the list → Stack / Recursion
18. Two Heaps 	Max-heap for lower half, min-heap for upper half; keep balanced.	- Running median - Split data into two halves - Best of both ends	- Find Median from Data Stream - Sliding Window Median - IPO - Meeting Rooms III - Process Tasks Using Servers	Time $O(\log n)$ add Space $O(n)$	<pre>// \$lo = SplMaxHeap, \$hi = SplMinHeap \$lo->insert(\$x); \$hi->insert(\$lo->extract()); // balance if (\$hi->count() > \$lo->count()) \$lo->insert(\$hi->extract()); \$med = \$lo->count() > \$hi->count() ? \$lo->top() : (\$lo->top() + \$hi->top()) / 2;</pre>	△ Forgetting to rebalance △ SplHeap can't delete arbitrary → lazy delete △ Even count → average of two tops	X Data is static → Sort once, pick middle

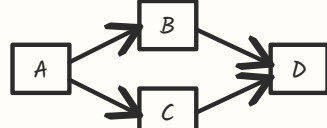
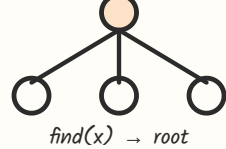
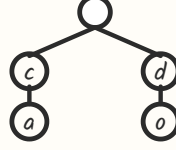
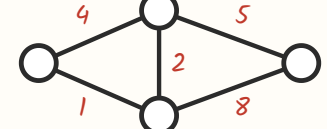
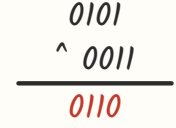
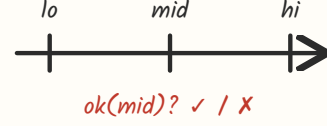
Top 5 = problems most commonly asked for each pattern (Blind 75 / NeetCode 150 / LeetCode company-tag favourites).



⇒ DSA PATTERN RECOGNITION — COMPLETE SHEET

RIGHT APPROACH · PHP EDITION

Part 4 of 4 · Patterns 19–24

PATTERNS	CORE IDEA	SPOT IT FROM	TOP 5 ASKED	TIME / SPACE	KEY TEMPLATE (PHP)	COMMON TRAPS	NOT THIS IF → USE
19. Topological Sort 	Kahn's BFS: repeatedly take nodes with in-degree 0.	- Dependencies / prerequisites - Order of tasks / build - Cycle in directed graph	- Course Schedule - Course Schedule II - Alien Dictionary - Minimum Height Trees - Parallel Courses	Time $O(V+E)$ Space $O(V+E)$	<pre>\$q = new SplQueue(); \$order = []; foreach (\$indeg as \$u => \$d) if (\$d === 0) \$q->enqueue(\$u); while (!\$q->isEmpty()) { \$u = \$q->dequeue(); \$order[] = \$u; foreach (\$adj[\$u] ?? [] as \$v) if (--\$indeg[\$v] === 0) \$q->enqueue(\$v); } \$hasCycle = count(\$order) < \$n;</pre>	△ Edge direction reversed △ Init indegree 0 for every node △ Not checking for a cycle	✗ Graph is undirected → Union Find / DFS
20. Union Find (DSU) 	Group elements into sets; union + find with path compression.	- Connected components - Dynamic connectivity - Cycle in undirected graph	- Number of Provinces - Redundant Connection - Accounts Merge - Graph Valid Tree - No. of Connected Components	Time $\approx O(\alpha(n))$ per op Space $O(n)$	<pre>function find(array &\$p, int \$x): int { if (\$p[\$x] !== \$x) \$p[\$x] = find(\$p, \$p[\$x]); return \$p[\$x]; } function unite(array &\$p, int \$a, int \$b): bool { \$ra = find(\$p, \$a); \$rb = find(\$p, \$b); if (\$ra === \$rb) return false; // cycle \$p[\$ra] = \$rb; return true; }</pre>	△ Comparing $\\$x$ instead of $find(\\$x)$ △ No path compression → slow △ Forgot $\\$p[\\$i] = \\$i$ init	✗ Need the actual path / distance → BFS / DFS ✗ Directed graph → Topological Sort
21. Trie (Prefix Tree) 	Tree of characters; each path from root spells a prefix.	- Prefix search / autocomplete - Word dictionary - Many words in a grid	- Implement Trie - Design Add and Search Words - Word Search II - Replace Words - Search Suggestions System	Time $O(L)$ per word Space $O(\text{total chars})$	<pre>class Node { public array \$kids = []; public bool \$end = false; } \$node = \$root; foreach (str_split(\$word) as \$ch) { \$node->kids[\$ch] ??= new Node(); \$node = \$node->kids[\$ch]; } \$node->end = true;</pre>	△ Missing end flag (prefix ≠ word) △ <code>str_split</code> breaks UTF-8 → <code>mb_str_split</code> △ Word Search II: prune found words	✗ Exact lookups only, no prefixes → Hash Set
22. Dijkstra (Weighted) 	BFS with a min-heap on distance; relax edges greedily.	- Shortest path, weights ≥ 0 - Min cost / time / effort - Network delay	- Network Delay Time - Path With Minimum Effort - Cheapest Flights Within K Stops - Swim in Rising Water - Path with Maximum Probability	Time $O((V+E) \log V)$ Space $O(V+E)$	<pre>\$dist = array_fill(0, \$n, PHP_INT_MAX); \$dist[\$src] = 0; \$pq = new SplPriorityQueue(); // max-heap \$pq->setExtractFlags(SplPriorityQueue::EXTR_BOTH); \$pq->insert(\$src, 0); // priority = -dist while (!\$pq->isEmpty()) { ['data' => \$u, 'priority' => \$p] = \$pq->extract(); if (-\$p > \$dist[\$u]) continue; // stale foreach (\$adj[\$u] as [\$v, \$w]) if (\$dist[\$u] + \$w < \$dist[\$v]) { \$dist[\$v] = \$dist[\$u] + \$w; \$pq->insert(\$v, -\$dist[\$v]); } }</pre>	△ Missing the stale-entry check △ Negative weights break it △ Forgot to negate priority (max-heap)	✗ Negative edges / $\leq K$ stops → Bellman-Ford ✗ Unweighted graph → BFS
23. Bit Manipulation 	Use XOR / AND / shifts; $x \wedge x = 0$, $x \& (x-1)$ drops lowest bit.	- Odd one out / single number - Subsets as bitmask - Power of two / count bits	- Single Number - Number of 1 Bits - Counting Bits - Reverse Bits - Sum of Two Integers	Time $O(n)$ Space $O(1)$	<pre>(\$x ^ \$x) === 0 // pairs cancel \$x & (\$x - 1) // drop lowest set bit \$x & ~\$x // lowest set bit (\$mask >> \$i) & 1 // is bit i set? \$r = 0; foreach (\$a as \$x) \$r ^= \$x; // single no. for (\$m = 0; \$m < (1 << \$n); \$m++) { /* subset */ }</pre>	△ $\wedge$ & bind weaker than $===$ → brackets △ PHP ints are 64-bit, no 32-bit wrap △ $>>$ on negatives keeps the sign	✗ $n > 20$ for bitmask subsets → DP / Greedy
24. Binary Search on Answer 	Guess the answer, check if feasible, shrink the range.	"Minimize the maximum" "Maximize the minimum" - Yes / no check is monotonic	- Koko Eating Bananas - Capacity To Ship Packages in D Days - Split Array Largest Sum - Min Days to Make m Bouquets - Magnetic Force Between Two Balls	Time $O(n \log \text{range})$ Space $O(1)$	<pre>\$lo = 1; \$hi = max(\$piles); while (\$lo < \$hi) { \$mid = intval(\$lo + \$hi, 2); if (ok(\$mid)) \$hi = \$mid; // try smaller else \$lo = \$mid + 1; } return \$lo;</pre>	△ Wrong bounds (ship: $lo = \max(w)$) △ <code>ok()</code> must be monotonic △ $\\$lo = \\mid with $\\$lo < \\hi → loop	✗ Searching an element's index → Binary Search (#5)

Top 5 = problems most commonly asked for each pattern (Blind 75 / NeetCode 150 / LeetCode company-tag favourites).

